

Universidad Tecnológica Nacional
Facultad Regional San Nicolás
Tecnicatura Universitaria en Programación a Distancia (TUPaD)
Programación 1

Gestión de Datos de Países en Python

Integrantes:

Gonzalo Ruiz
Agustín Fassi

Fecha de entrega: 17 de junio de 2026

Índice

1. Marco Teórico.....	3
1.1. Listas y diccionarios en Python.....	3
1.2. Funciones y modularización.....	3
1.3. Manejo de archivos CSV	3
1.4. Estructuras condicionales y repetitivas	4
1.5. Ordenamiento y estadísticas.....	4
2. Decisiones Técnicas y Arquitectura.....	5
2.1. Estructura del programa	5
2.2. Diagrama de flujo.....	5
2.3. Capturas de ejecución	6
3. Dificultades y Conclusiones	8
4. Bibliografía / Webgrafía	9
5. Links del Proyecto	10

1. Marco Teórico

1.1. Listas y diccionarios en Python

Las listas son estructuras de datos ordenadas y mutables que permiten almacenar colecciones de elementos. En Python se definen con corchetes y admiten elementos de cualquier tipo. Son ideales para representar conjuntos de registros donde el orden de inserción importa.

Los diccionarios son estructuras de datos que asocian claves únicas a valores. Cada entrada del dataset de países se representa como un diccionario donde las claves son los nombres de las columnas (nombre, poblacion, superficie, continente) y los valores son los datos correspondientes a cada país. Esta estructura permite acceder a los datos de forma clara y explícita mediante su clave, lo que facilita la lectura y el mantenimiento del código.

1.2. Funciones y modularización

La modularización es el proceso de dividir un programa en partes más pequeñas y manejables llamadas funciones. Cada función tiene una responsabilidad específica, recibe parámetros de entrada y devuelve un resultado. Este enfoque facilita la lectura del código, permite reutilizar lógica en distintas partes del programa y simplifica la detección de errores.

En este proyecto se optó por centralizar todo el código en un único archivo Python, organizando las funciones en secciones claramente delimitadas mediante comentarios: sección de carga y guardado de datos, sección de visualización, sección de altas y modificaciones, sección de consultas y filtros, sección de ordenamiento, sección de estadísticas y menú principal.

1.3. Manejo de archivos CSV

Un archivo CSV (Comma-Separated Values) es un formato de texto plano donde cada fila representa un registro y los campos están separados por comas. Python incluye el módulo csv en su biblioteca estándar, que provee la clase DictReader para leer archivos CSV directamente como listas de diccionarios, asignando automáticamente los nombres de las columnas como claves, y la clase DictWriter para escribir esas listas de diccionarios nuevamente al archivo, manteniendo el mismo formato de columnas.

1.4. Estructuras condicionales y repetitivas

Las estructuras condicionales (if / elif / else) permiten ejecutar diferentes bloques de código según el valor de una condición booleana. Se utilizan en este proyecto para validar entradas del usuario, filtrar resultados y gestionar casos especiales como búsquedas sin resultados.

Las estructuras repetitivas (for / while) permiten iterar sobre colecciones de datos o repetir acciones hasta que se cumpla una condición. El bucle while sostiene el menú principal activo hasta que el usuario elige la opción de salir, mientras que los bucles for recorren la lista de países para mostrar registros, filtrar, ordenar o calcular estadísticas.

1.5. Ordenamiento y estadísticas

Python ofrece la función integrada `sorted()` que permite ordenar listas de forma ascendente o descendente. Combinada con el parámetro `key` y funciones `lambda`, es posible ordenar listas de diccionarios por cualquier campo numérico o textual, como nombre, población o superficie.

Para el cálculo de indicadores estadísticos se utilizaron funciones como `max()` y `min()` junto con la función `sum()` dentro de expresiones generadoras, lo que permite obtener el país más poblado, el país menos poblado, el promedio de población, el promedio de superficie y la distribución de países por continente.

2. Decisiones Técnicas y Arquitectura

2.1. Estructura del programa

Se decidió trabajar con un único archivo Python ("gestion_paises.py") acompañado del dataset ("paises.csv"), cuyas columnas son nombre, poblacion, superficie y continente. Dentro del archivo, el código está organizado en cinco secciones funcionales claramente separadas:

Sección	Responsable	Funciones principales
Datos	Ambos	cargar_paises(), guardar_paises()
Visualización y registro	Ambos	mostrar_paises(), agregar_pais(), actualizar_pais()
Consultas y filtros	Agustín	buscar_por_nombre(), filtrar_por_continente(), filtrar_por_rango_poblacion(), filtrar_por_rango_superficie()
Ordenamiento y estadísticas	Gonzalo	ordenar_paises(), mostrar_estadisticas()
Menú principal	Ambos	menu()

Esta organización permite que cada integrante trabaje de forma independiente sobre su sección sin generar conflictos, y facilita la revisión del código al momento de hacer los commits en el repositorio. Los cambios producidos por las operaciones de alta y actualización se persisten de inmediato llamando a guardar_paises(), por lo que el archivo paises.csv siempre refleja el estado actual de la lista en memoria.

2.2. Diagrama de flujo

A continuación se describe el flujo principal de ejecución del programa:

Paso	Descripción
1	El programa inicia y llama a cargar_paises() para leer el archivo paises.csv
2	Se muestra el menú principal con las 10 opciones disponibles (1 a 9 y 0 para salir)
3	El usuario ingresa una opción numérica
4a	Si elige mostrar, buscar, filtrar u ordenar (opciones 1, 4 a 8) → se llama a la función correspondiente, que opera sobre la lista en memoria y muestra el resultado con mostrar_paises()
4b	Si elige agregar o actualizar (opciones 2 o 3) → se llama a agregar_pais() o actualizar_pais(), que validan los datos ingresados y llaman a guardar_paises() para persistir los cambios en paises.csv
4c	Si elige ver estadísticas (opción 9) → se llama a mostrar_estadisticas()
5	Se muestran los resultados en pantalla

6	Se vuelve al menú (paso 2) hasta que el usuario elige la opción 0, momento en el cual el programa finaliza
---	--

2.3. Capturas de ejecución

A continuación se muestra un ejemplo representativo de la ejecución del programa en consola.

Menú principal:

```
=====
          GESTIÓN DE PAÍSES
=====
1. Mostrar todos los países
2. Agregar país
3. Actualizar país
4. Buscar por nombre
5. Filtrar por continente
6. Filtrar por rango de población
7. Filtrar por rango de superficie
8. Ordenar países
9. Ver estadísticas
0. Salir
=====
Opción:
```

Resultado de búsqueda por nombre 'Argentina' (opción 4):

```
-- Buscar por nombre --
Ingresá el nombre o parte del nombre: Argentina

Nombre                Población    Superficie (km²) Continente
-----
Argentina              46,000,000      2,780,400 America del Sur
```

Estadísticas generales (opción 9):

```
-- Estadísticas --

Población:
País con mayor población : India (1,428,000,000)
País con menor población : Nueva Zelanda (5,000,000)
Promedio de población    : 754,750,000

Superficie:
Promedio de superficie   : 3,262,120 km²

Países por continente:
America del Sur: 1 país/es
Asia           : 3 país/es
Oceania        : 1 país/es
```

La salida muestra el ejemplo con un dataset de cinco países (Argentina, China, Japón, India y Nueva Zelanda). Los valores numéricos se imprimen con separador de miles para facilitar su lectura.

3. Dificultades y Conclusiones

Dificultades encontradas

Una de las primeras dificultades fue comprender cómo leer el archivo CSV y transformarlo en una lista de diccionarios utilizable. Al principio intentamos acceder a los datos de forma manual con índices numéricos, hasta que entendimos cómo funciona `csv.DictReader` y que permite acceder a cada campo por su nombre de columna, lo que simplificó mucho el código.

Otro obstáculo fue el manejo de los tipos de datos: los valores leídos desde el CSV llegan como cadenas de texto, por lo que al intentar comparar o calcular estadísticas numéricas (como población o superficie) obteníamos resultados incorrectos. Tuvimos que aprender a convertir esos valores con `int()` al momento de cargar los datos.

También nos costó al principio organizar el menú principal con el bucle `while` de forma que no se rompa ante entradas inválidas del usuario. La solución fue validar la opción ingresada contra el rango de opciones disponibles (0 a 9) e informar al usuario cuando ingresa un valor fuera de ese rango, sin interrumpir el bucle principal.

Por último, escribir el archivo CSV cada vez que se agrega o actualiza un país nos obligó a prestar atención al orden y nombre de las columnas en `guardar_paises()`, ya que cualquier discrepancia con los encabezados originales podía generar inconsistencias al volver a cargar el archivo en la siguiente ejecución.

Conclusiones

Este trabajo nos permitió ver cómo los conceptos aprendidos en clase se aplican juntos para construir un programa real. Entendimos que no alcanza con saber usar listas o funciones por separado, sino que la clave está en combinarlos de forma ordenada.

Trabajar en equipo implicó acordar cómo nombrar las variables y funciones para que el código de cada uno encajara con el del otro. Esto nos hizo valorar la importancia de los comentarios y de mantener un estilo consistente en todo el archivo.

En definitiva, el proyecto consolidó nuestra comprensión de Python como herramienta para resolver problemas concretos de manejo de información, y nos dejó una base sólida para encarar trabajos más complejos en los próximos cuatrimestres.

4. Bibliografía / Webgrafía

- [1] Python Software Foundation. (2024). csv — CSV File Reading and Writing. Documentación oficial de Python 3. <https://docs.python.org/3/library/csv.html>
- [2] Python Software Foundation. (2024). Built-in Functions. Documentación oficial de Python 3. <https://docs.python.org/3/library/functions.html>
- [3] Python Software Foundation. (2024). Data Structures — Python Tutorial. <https://docs.python.org/3/tutorial/datastructures.html>
- [4] Python Software Foundation. (2024). Sorting HOW TO. <https://docs.python.org/3/howto/sorting.html>
- [5] Severance, C. (2016). Python for Everybody: Exploring Data in Python 3. CreateSpace Independent Publishing Platform.

5. Links del Proyecto

Repositorio en GitHub

Link: <https://github.com/gowonza-coder/Trabajo-Integrador-Obligatorio---Programaci-n-1>

El repositorio contiene el código fuente completo del proyecto (gestion_paises.py) junto al dataset (paises.csv).

Video explicativo

Link: <https://www.youtube.com/watch?v=NE3Tol4aeOA>

El video presenta una demostración en vivo del programa en ejecución, explicando las decisiones técnicas tomadas y recorriendo las funcionalidades principales del sistema.